

Atlas: motor relacional en disco

Gestor de Bases de Datos Multimodal Informe técnico del Entregable 1

Equipo del proyecto / CS2042

19 de septiembre de 2026

Resumen

Se implementa un motor relacional sobre archivos binarios paginados: Heap con lista libre, Sequential con overflow enlazado, árbol B+ multinivel, hashing extensible y un subconjunto de SQL. FastAPI expone el motor a un cliente web con métricas de transferencias de páginas. Se valida la persistencia de 500 000 productos y se ejecutan cuatro experimentos sobre estructuras reales. El diseño prioriza algoritmos explícitos y memoria acotada durante el procesamiento de datos.

Índice

1. Introducción y alcance	2
2. Diseño físico y medición	2
2.1. Page Layout y RID	2
2.2. Serialización y contador	2
3. Organización de archivos	2
3.1. Heap y Free-List	2
3.2. Sequential, overflow y reorganización	3
4. Índices persistentes	3
4.1. Árbol B+	3
4.2. Extendible Hashing	3
4.3. Integridad y catálogo	4
5. SQL, API y cliente web	4
6. Experimentación y rendimiento	5
6.1. Método y reproducibilidad	5
6.2. Validación del CSV después de reabrir	5
6.3. Experimento 1: costo de inserción masiva	6
6.4. Experimento 2: igualdad	7
6.5. Experimento 3: selectividad de rangos	8
6.6. Experimento 4: sensibilidad al tamaño de página	9
7. Conclusiones y límites	10

1. Introducción y alcance

El proyecto materializa los componentes del Entregable 1 del enunciado de CS2042, período 2026-II. Las carpetas del repositorio original se conservan y se completan; se separan representación de páginas, serialización de registros, algoritmos de índices y ejecución SQL. No se emplea ningún motor de almacenamiento externo ni serialización de objetos Python para guardar tablas o índices.

El dataset proporcionado, `products-500000.csv`, contiene exactamente 500 000 filas y 14 columnas. La columna `Index` se mapea a `product_id`; sus claves son enteros únicos crecientes entre 1 y 500 000. El tamaño original es 90 764 217 bytes. Su procedencia externa no se presume: se utiliza el archivo recibido del usuario como dataset sintético del proyecto.

Se verificaron las longitudes máximas de todos los campos en bytes UTF-8 antes de definir los tamaños CHAR. Cada producto ocupa 394 bytes. EAN se almacena como texto para conservar sus ceros iniciales; Added Date permanece como texto porque el subconjunto solicitado no define DATE. Los módulos espaciales, textuales y vectoriales pertenecen al Entregable 2 y no se implementan aquí.

2. Diseño físico y medición

2.1. Page Layout y RID

Toda transferencia binaria pasa por `DiskManager`. Los tamaños permitidos son 1024, 2048, 4096 y 8192 bytes, con 4096 por defecto. El desplazamiento físico de una página se calcula como:

$$\text{offset} = \text{page_id} \times \text{page_size}.$$

La cabecera común ocupa 20 bytes y contiene identificador, número de registros, offset libre y enlaces siguiente/anterior. En páginas de registros se agregan 12 bytes de configuración, metadata opcional, un byte de presencia por slot y los registros de longitud fija. Si m es la metadata adicional y r el tamaño del registro, la capacidad es:

$$C = \left\lfloor \frac{B - 20 - 12 - m}{r + 1} \right\rfloor.$$

Para productos en Heap, $B = 4096$, $r = 394$, $m = 0$: caben diez registros por página. El primer registro comienza en el byte 42. La página cero contiene metadata del archivo y no forma parte de las filas de la tabla.

El RID es el par `(page_id, slot_number)`. Las eliminaciones liberan slots sin mover otros registros. Para distinguir el overflow del Sequential dentro del mismo espacio lógico de RID, una página de overflow p se codifica como $-p - 1$; los identificadores físicos de sus archivos siguen siendo positivos.

2.2. Serialización y contador

`RecordSerializer` usa enteros y flotantes de 64 bits en formato little-endian, y CHAR de longitud fija en bytes UTF-8. Rechaza valores de tipo incorrecto, texto demasiado largo, caracteres NUL y flotantes no finitos.

`DiskCounter` incrementa una lectura después de una transferencia completa de página y una escritura después de una escritura completa, incluida la asignación inicial. No se usan fórmulas para fabricar estas métricas. El contador se reinicia por consulta e incluye las páginas de metadata abiertas por el ejecutor. El catálogo JSON es metadata administrativa y no entra en el contador de páginas.

El archivo se abre sin buffering de Python. Las cifras cuentan transferencias solicitadas al archivo, no fallos de caché del sistema operativo ni accesos mecánicos del dispositivo. El SO puede retener páginas; no se fuerza caché fría ni `fsync` por operación. Para recuperar varios RID, el ejecutor retiene como máximo 32 páginas de datos por consulta; las consultas comienzan sin ese buffer.

3. Organización de archivos

3.1. Heap y Free-List

La metadata del Heap contiene el tamaño de registro, cantidad de filas y `free_head`. Esta cabecera apunta a la primera página que dispone de slots. Cada página libre enlaza la siguiente mediante `next_page_id`.

Insertar lee únicamente la cabecera conocida de la lista y su primera página, o anexa una página nueva: el costo de I/O de la organización es constante. La comprobación de unicidad y el mantenimiento de índices son costos adicionales del ejecutor.

La eliminación es lógica mediante el bitmap. Si una página antes llena recupera un slot, se añade a la lista libre. El escaneo recorre todas las páginas de datos, con una lectura por página, y devuelve solamente slots activos.

3.2. Sequential, overflow y reorganización

Las páginas principales y sus registros activos están ordenados por la clave primaria. Cada página persiste una frontera inferior que no cambia al borrar registros; esto permite buscar páginas mediante búsqueda binaria incluso si una página queda vacía. Insertar ocupa un slot únicamente cuando preserva el orden físico sin desplazar RID. Las inserciones ascendentes pueden anexar nuevas páginas.

Cuando no hay un hueco apropiado, el registro se inserta en una lista de overflow ordenada, cuya cabeza pertenece a la página principal correspondiente. Cada registro de overflow almacena el RID siguiente. `OverflowManager` reutiliza un Heap para asignar slots, pero el orden lógico reside en los enlaces binarios, no en una lista en RAM.

`reorganize()` recorre las páginas principales y sus cadenas, fusiona ambos flujos ordenados y escribe un archivo principal temporal. Llena cada página hasta aproximadamente 75 % (redondeado hacia abajo), reemplaza el principal y reinicia el overflow. Conserva solo páginas y cursores en memoria. El ejecutor reconstruye todos los índices, incluido el de integridad de la clave primaria, porque los RID cambian durante esta operación.

Un overflow muy largo incrementa el costo de inserción y búsqueda. La reorganización es mantenimiento explícito mediante API o Python; no hay un planificador periódico automático. No se requiere ordenar toda la tabla en memoria para reorganizarla.

4. Índices persistentes

4.1. Árbol B+

`bplus_node.py` representa y serializa nodos; `bplus_tree.py` implementa descenso, inserción y splits. Los internos contienen punteros a hijos y separadores; las hojas contienen pares clave/RID y enlaces bidireccionales. Se usa el par compuesto clave/RID para ordenar claves secundarias repetidas sin perder registros.

`_find_leaf()` desciende desde la raíz leyendo nodos y conserva el camino de padres. Al desbordar una hoja, `_split_leaf()` separa aproximadamente la mitad y copia al padre la primera clave derecha. Si desborda un interno, se promueve su separador central, se divide y se continúa hacia arriba. Si no hay padre se crea una nueva raíz y se persisten su identificador y altura.

Los rangos localizan la hoja inicial, avanzan por `next_leaf_id` y se detienen al superar el máximo. La capacidad depende del tamaño real de página y del tipo de clave. Se utiliza un máximo común de claves que cabe tanto en hojas como en internos. El borrado elimina el par exacto sin fusionar nodos; los límites de separación siguen siendo válidos. Las hojas vacías permanecen enlazadas.

4.2. Extendible Hashing

El índice mantiene buckets con profundidad local y un directorio paginado con profundidad global. `hash_directory.py` lee bloques de punteros; el directorio completo nunca se materializa en RAM. Un hash BLAKE2 de 64 bits produce los mismos bits al reabrir el proceso, a diferencia de `hash(str)` de Python. El prefijo utilizado son los bits bajos según la profundidad global.

Si la profundidad local del bucket lleno coincide con la global, se duplica el directorio físicamente. Luego se divide el bucket, aumenta su profundidad local, se redistribuyen entradas según el nuevo bit y se actualizan las referencias correspondientes. El split también funciona sin duplicación cuando ya existe suficiente profundidad global.

Para claves repetidas o colisiones de hash indistinguibles se usan páginas encadenadas; la profundidad máxima es 20 para limitar el tamaño del directorio. No hay contracción ni merge de buckets. Las páginas vacías de colisiones retiradas en un split no se reciclan; reconstruir el índice recupera ese espacio.

4.3. Integridad y catálogo

El catálogo JSON conserva esquemas, organización, tamaño de página y nombres de archivos e índices. Cada tabla incluye un Hash interno para comprobar la unicidad de PRIMARY KEY; el planificador no utiliza ese índice privado como ruta de SELECT. Esto permite demostrar un Full Scan real antes de crear índices públicos, sin volver cuadrática la importación. Sus lecturas y escrituras sí forman parte de las métricas del ejecutor.

5. SQL, API y cliente web

El tokenizer propio distingue identificadores, operadores, literales, comentarios y cadenas con comillas simples escapadas. El parser produce `ParsedQuery`; acepta CREATE TABLE, INSERT INTO VALUES, CREATE INDEX, SELECT y DELETE con WHERE. Los tipos son INT, FLOAT y CHAR(n). Los filtros combinan comparaciones mediante AND. No se implementan JOIN, UPDATE, agregaciones, OR ni SQL completo.

Las reglas del planificador son explícitas: igualdad con índice público aplicable utiliza IndexScan; rango con B+ utiliza IndexRangeScan; el resto utiliza SeqScan. Cuando hay B+ y Hash sobre una igualdad, se prefiere Hash. Hash nunca atiende rangos. En Sequential, el escaneo de la organización puede acotarse por la búsqueda binaria de la clave primaria aunque la ruta no use un índice secundario. Todos los predicados se vuelven a evaluar sobre los registros recuperados.

La API ofrece POST `/api/query`, GET `/api/tables` y POST `/api/tables/reorganize`. Devuelve filas, ruta de acceso, índice, tiempos de parseo y ejecución, lecturas y escrituras. Se emplea un lock local para serializar solicitudes concurrentes del único proceso del servidor.

El cliente HTML/CSS/JavaScript se sirve en el mismo origen mediante FastAPI. Integra explorador de esquemas e índices, editor con resaltado SQL, resultados paginados y panel de métricas con historial gráfico. El servidor conserva como máximo 1000 filas en cada respuesta y cuenta el total recorriendo la consulta. Cada cambio de página ejecuta de nuevo el SQL y muestra sus nuevas métricas. Se verificaron consultas reales, paginación y errores en tres anchuras de ventana, incluyendo 1366 y 1920 píxeles.

6. Experimentación y rendimiento

6.1. Método y reproducibilidad

Todos los experimentos leen el mismo CSV con `csv.DictReader` y lo convierten mediante el serializador del motor. Se conserva el orden ascendente de `product_id`. La carga de índices B+ y Hash del experimento 1 incluye un Heap de productos completos y su índice; no se comparan únicamente pares clave/RID contra registros completos. Sus tiempos incluyen lectura del CSV, serialización y transferencias de archivos. No se incluye el Hash privado de unicidad del ejecutor en estas mediciones de estructuras aisladas.

Los experimentos 2 y 3 comparten un fixture persistente de 100 000 productos. El Sequential se reorganiza a 75 % antes de las búsquedas. Los índices apuntan a los RID reales del Heap y se cuenta la recuperación del registro, no solo el acceso al índice. El experimento 4 aísla el árbol B+ y no recupera payloads; sus RID son etiquetas de prueba. Las claves de búsqueda se generan con semilla 42. La desviación estándar reportada es poblacional.

Las cifras provienen de operaciones reales, no de costos estimados. El SO mantiene su caché y la latencia depende del equipo y carga del sistema; pequeñas diferencias de tiempo entre índices no prueban superioridad general. La comparación de transferencias es más estable. Los resultados crudos, resúmenes CSV y gráficas se guardan en `benchmarks/results/` y `benchmarks/plots/`. Los experimentos se ejecutan secuencialmente entre sí.

6.2. Validación del CSV después de reabrir

Tabla 1: Importación integrada con comprobación de PRIMARY KEY. Creación de tabla medida por separado.

Filas	Verificadas	Tiempo (s)	Lecturas	Escrituras
1 000	1 000	0.40	5 934	4 032
10 000	10 000	4.79	64 946	45 944
100 000	100 000	51.71	621 088	431 086
500 000	500 000	323.00	3 108 130	2 153 577

La comparación recorre nuevamente el CSV y el Heap reabierto; exige igualdad de los bytes serializados de cada fila y comprueba los extremos del dominio. Los resultados completos incluyen lecturas/escrituras de creación en el JSON de validación.

6.3. Experimento 1: costo de inserción masiva

Tabla 2: Tiempo de carga por estructura, en segundos.

N	Heap	Seq.	Seq.+reorg.	Heap+B+	Heap+Hash
1 000	0.04	0.05	0.06	0.21	0.30
10 000	0.38	0.55	0.60	2.48	3.64
50 000	1.90	2.70	2.95	14.72	17.75
100 000	3.84	5.39	5.92	30.42	36.80
250 000	9.52	13.44	14.90	78.22	102.69
500 000	19.11	26.96	29.72	161.69	213.45

Tabla 3: Escrituras reales de páginas por carga.

N	Heap	Seq.	Seq.+reorg.	Heap+B+	Heap+Hash
1 000	2 001	2 101	2 246	3 020	4 037
10 000	20 001	21 001	22 432	30 198	45 949
50 000	100 001	105 001	112 146	150 989	215 297
100 000	200 001	210 001	224 289	301 979	431 091
250 000	500 001	525 001	560 718	754 947	1 080 015
500 000	1 000 001	1 050 001	1 121 432	1 509 897	2 153 582

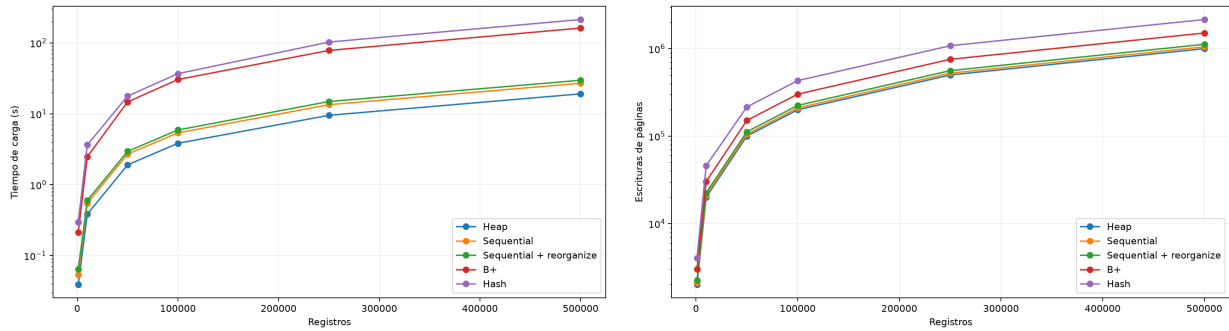


Figura 1: Cargas completas: tiempo y escrituras, con eje vertical logarítmico.

Heap escribe la página del registro y su metadata en cada inserción. Sequential aprovecha el CSV ascendente; reorganizar añade la lectura del principal y su reescritura con espacios libres. B+ y Hash incorporan mantenimiento del índice además del payload. La cantidad de escrituras incluye inicialización y splits reales. Este workload no mide el peor caso de overflow desordenado.

6.4. Experimento 2: igualdad

Tabla 4: 1000 consultas de igualdad sobre 100000 productos; incluye recuperar la fila.

Estructura	Media I/O	Desv. I/O	Media ms	Desv. ms
Heap	10 000.000	0.000	161.0724	3.4962
Sequential	14.971	0.508	0.1910	0.0352
B+	4.022	0.147	0.1515	0.0335
Hash	3.000	0.000	0.1614	0.0331

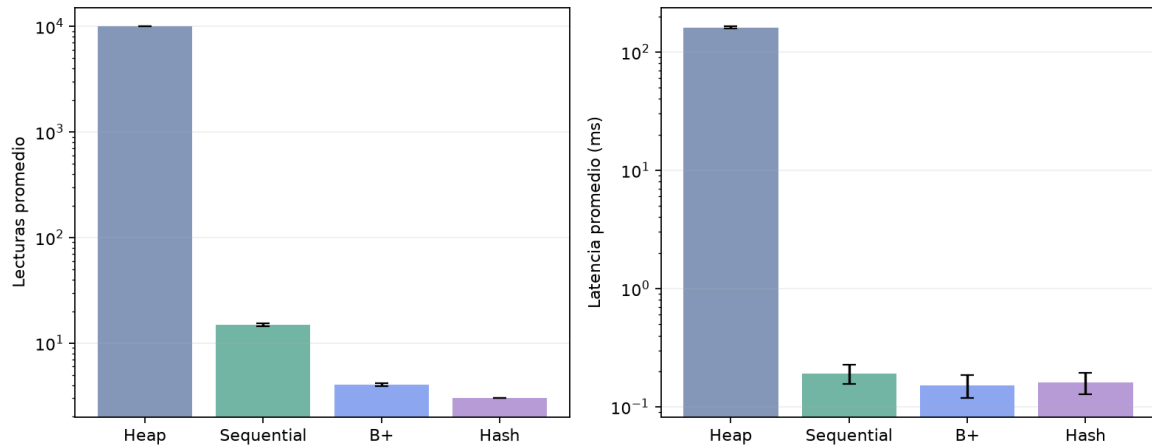


Figura 2: Media y desviación estándar poblacional; escala logarítmica.

El Full Scan lee las 10000 páginas de productos en cada consulta. Sequential necesita la búsqueda binaria y la página candidata. El B+ desciende desde la raíz y recupera el payload del Heap; algunas consultas leen una hoja adicional para cerrar el rango de igualdad. Hash lee una página del directorio, un bucket y una página de datos. La desviación nula de I/O del Hash observado indica ausencia de cadenas para estas claves únicas; no es una garantía para cualquier distribución.

6.5. Experimento 3: selectividad de rangos

Tabla 5: Rangos inclusivos centrados en el dominio; cardinalidad verificada en cada consulta.

Sel. (%)	Heap I/O	Seq. I/O	B+ I/O	Heap ms	Seq. ms	B+ ms
0.1	10 000	28	15	163.05	0.47	0.97
1.0	10 000	158	114	160.82	2.71	2.11
5.0	10 000	728	553	160.26	12.44	9.77
10.0	10 000	1 444	1 102	162.85	26.20	20.26
25.0	10 000	3 586	2 749	163.15	63.29	50.78

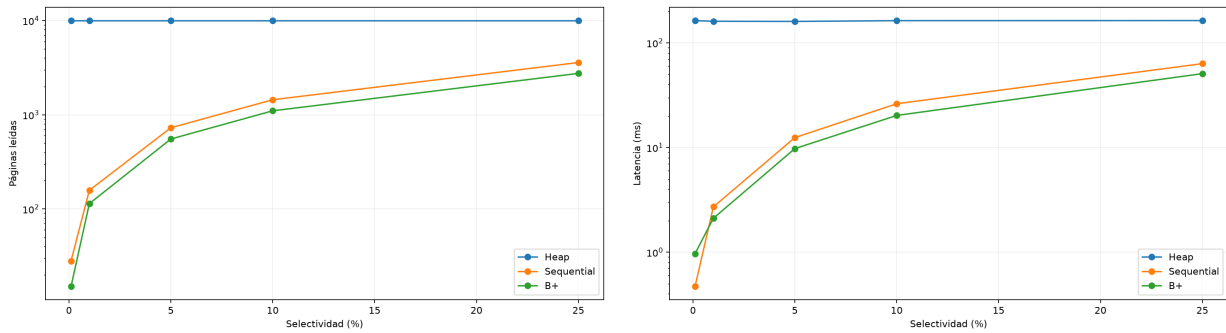


Figura 3: Costo de rangos según selectividad, con escala vertical logarítmica.

Heap mantiene un costo de escaneo completo independientemente de cuántas filas satisfacen el predicado. Sequential y B+ incrementan sus lecturas a medida que el rango necesita más páginas. El Sequential reorganizado contiene siete productos por página, mientras el Heap de payloads del B+ contiene diez; parte de la diferencia procede de esta ocupación, además del costo de navegar el índice. Las consultas del B+ reutilizan una página de datos durante la recuperación de RID consecutivos.

6.6. Experimento 4: sensibilidad al tamaño de página

Tabla 6: Árbol B+ con 100000 claves; 1000 búsquedas. No incluye recuperar productos.

Bytes	Fan-out	Altura	Páginas	I/O búsqueda	Escrituras carga
1 024	50	4	4 160	4.071	108 315
2 048	102	3	1 999	3.039	103 994
4 096	204	3	991	3.022	101 978
8 192	409	3	494	3.012	100 984

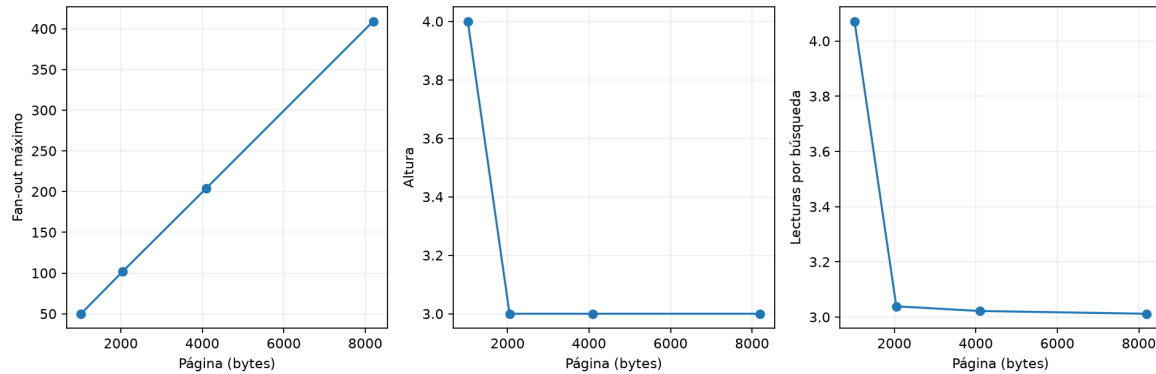


Figura 4: Estructura y lecturas del B+ para los cuatro tamaños de página.

Con 1024 bytes, el árbol requiere cuatro niveles; con los tamaños mayores, tres. Aumentar la capacidad reduce el número de nodos y las escrituras de splits. La latencia de construcción no disminuye necesariamente: el serializador procesa más entradas por nodo al aumentar el tamaño de página. La reducción de I/O debe distinguirse del costo de CPU y de la caché del SO.

7. Conclusiones y límites

La suite final aprobó 41 pruebas, con cero fallos. El motor persiste páginas, tablas e índices y recupera su estado al reabrir archivos sin reconstruir obligatoriamente los índices. Los tests verifican splits de hojas e internos, duplicación de directorio, búsqueda por hojas enlazadas, reutilización de slots, overflow y reorganización. Se verificaron 500 000 productos persistidos comparando los bytes de cada registro con el CSV original. Además, se construyeron y reabrieron índices públicos sobre ese volumen: se comprobaron las 500 000 entradas del B+ en orden y 1002 búsquedas por cada índice B+ y Hash.

Las búsquedas de igualdad muestran la diferencia esperada: el Heap recorre todas sus páginas, mientras el B+ depende principalmente de la altura y el Hash de directorio y bucket. Los rangos amplios requieren más páginas tanto en B+ como en Sequential. Las páginas grandes reducen la cantidad de nodos y pueden reducir la altura; sin embargo, serializar más entradas por nodo también puede aumentar el tiempo de CPU en Python.

La implementación es académica y de un solo proceso escritor. No incluye WAL, transacciones ACID, rollback de sentencias ni recuperación automática ante interrupción durante escrituras múltiples o reorganización. La reorganización reemplaza archivos pero no es una transacción atómica que abarque todos los índices. Debe evitarse ejecutar importadores simultáneamente con el servidor sobre el mismo directorio. Una importación interrumpida deja las filas ya insertadas; para repetirla se utiliza una tabla o directorio nuevos.

El B+ no fusiona nodos al borrar y el Hash no contrae el directorio. El overflow requiere mantenimiento explícito. CHAR(n) limita bytes, no caracteres Unicode. El subconjunto SQL requiere una PRIMARY KEY simple por tabla y no soporta NULL. Estos límites están documentados y no se presentan como funcionalidades implementadas.

La guía `EXPOSICION.md` relaciona cada algoritmo con sus archivos y funciones e incluye un guion de demostración. La grabación del equipo y sus datos personales de portada deben incorporarse al momento de la entrega académica.